GUIDE 27 · AGENTS

Check whether your MCP server is running, from the Console

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=27-check-the-mcp-server-from-the-console. This copy was made on 4 November 2026.

Time: 3 minutes. **You'll end up with:** a quick answer to "is the MCP server up, and what will agents see?" - without installing an MCP client.

`queryapigate mcp` is a separate process from the server that serves the Console, so the Console can't know about it on its own. The **MCP** screen in the sidebar asks on request - and lists the tools a client would see.

What the MCP screen shows

Connect a client - the config an MCP client needs: the server's URL and a scoped API key as the `X-API-Key` header - the same one-entry snippet you paste into Claude Desktop's config.

> The MCP server's own *settings* stay under **Settings → MCP server**, read-only: `QUERYAPIGATE_MCP_PORT` (default `5001`) and `QUERYAPIGATE_MCP_MAX_ROWS` (default `200`), with whether each comes from the environment or the default. These are the values the *Console's* server was started with - start both processes with the same variables (one `.env` file for both is easiest), or the reachability check uses the wrong port.

Reachability → Check now - tries a TCP connection to that port on `127.0.0.1`. It runs only when you press the button, never on its own. Verified both ways:

	RESULT
No MCP server running	*Not reachable on port 5001*
`queryapigate mcp` running on `QUERYAPIGATE_MCP_PORT=5061` (and the Console's server started with the same)	*Reachable on port 5061*

Tools - what `tools/list` returns for the admin key: each tool's name, kind (*saved query* or *ad-hoc*), parameters and description. Write queries never appear - they aren't exposed over MCP - so a query you expected but don't see here is probably a write.

The same over the API

```shell
curl http://127.0.0.1:5000/api/v1/mcp/status -H 'X-API-Key: demo-key'
# {"reachable": true, "port": 5061}

curl http://127.0.0.1:5000/api/v1/mcp/tools -H 'X-API-Key: demo-key'
# {"items": [{"name": "example_film_lookup", "description": "One film and how it is doing - for a
partner integration",
#             "kind": "saved query", "params": ["film_id"], "read_only": true}, ...,
#            {"name": "execute_sql", "kind": "ad-hoc", ...}]}
```

What "reachable" does and doesn't mean

- **Same host only.** The check connects to `127.0.0.1`. If the MCP server runs in another container or on another machine, it reports *Not reachable* even when it's fine. In Docker Compose, check it from its own container instead.
- **A port, not a protocol.** *Reachable* means something accepts connections on that port - not necessarily `queryapigate mcp`. For a real check, ask the MCP process itself: it serves `/health` on its own port.

  ```shell
  curl http://127.0.0.1:5061/health
  # {"status": "ok", "version": "0.13.0"}
  ```

- **The tool list is the admin key's.** A scoped key sees fewer tools - only what its grants reach, and `list_tables` / `execute_sql` only with a `connections` grant. To see exactly what an agent's key gets, list the tools with that key from an MCP client (see Let an AI agent call your saved queries via MCP), or call `GET /catalog` with it - the saved-query part matches, writes aside.

Seeing MCP traffic

MCP calls show up where REST ones do:

- **Run history** and the Home screen's live feed, with `"transport": "mcp"`.
- **The MCP process's own `/metrics`** (on its port, no key), with `method="MCP"` and endpoints `mcp.saved_query`, `mcp.execute_sql` and `mcp.list_tables`. The Console's Metrics screen shows the REST server's metrics only - scrape both processes to see everything (see Wire up Prometheus and Grafana).

Next steps

- Let an agent explore your schema and run ad-hoc SQL.
- Read structured MCP results properly.